

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 637 — Complexity Theory

Not whether it can be done, but what it costs — and why the central question has been open for fifty years

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 8 (with CSCE 627 and CSCE 658)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 627 Modules 01–06 for the machine model and reductions; probability for Module 08

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A written portfolio containing one NP-completeness proof you constructed, one honest account of how a hard problem is handled in practice, and a correct statement of what each result does and does not establish

Course description

CSCE 627 asked whether a problem can be solved at all. This course asks what it costs — and the difference is that **computability is settled and complexity is mostly not**. **The central question of the field has been open since 1971**, and a semester spent on a subject whose main question is unresolved teaches something about what evidence looks like when proof is unavailable.

The organising idea is that verification may be easier than search. Module 03 makes that precise as NP, and P versus NP is exactly the question of whether it is — which is a far more natural question than the formal statement suggests, and is why it matters outside the field.

The second theme is that the field is honest about its own obstacles. Module 07 covers relativisation and Module 09 covers natural proofs — two results showing that whole classes of proof technique cannot settle P versus NP. **Knowing why a question is hard is a genuine form of progress**, and few fields can point to theorems about the inadequacy of their own methods.

The third is that the practical consequences are not what the headline suggests. NP-hardness does not mean intractable — CSCE 669 Module 09's solvers dispatch NP-hard problems with millions of variables daily. **Modules 11 and 12 are about what the theory actually predicts**: which approximation ratios are achievable, which parameters make a problem easy, and why some polynomial algorithms are too slow and some exponential ones are fine.

And the closing position is that the classification is a starting point rather than a verdict. 'NP-complete' is the beginning of the engineering, not the end of it, and Module 13 says what the label

does and does not license you to conclude.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain why polynomial time is the robust notion of tractability.
- 2 Define NP by verification and by non-determinism, and relate them.
- 3 Prove a problem NP-complete by reduction.
- 4 Place problems in co-NP, PSPACE, and the polynomial hierarchy.
- 5 Explain space complexity and Savitch's theorem.
- 6 Prove a hierarchy theorem and explain the relativisation barrier.
- 7 Define BPP and explain the derandomisation question.
- 8 Explain circuit complexity and the natural proofs barrier.
- 9 Explain the PCP theorem and its approximation consequences.
- 10 Apply parameterised and fine-grained complexity to a real problem.
- 11 State what a hardness result does and does not establish.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Measuring Cost	How do you measure the cost of a computation so the answer means something?
2	P, and What Tractable Means	Why is polynomial time the dividing line?
3	NP, and Verification	Is checking an answer easier than finding one?
4	NP-Completeness	How can thousands of unrelated problems have exactly the same difficulty?
5	Beyond NP	What is harder than NP, and how would you know?
6	Space	What changes when memory rather than time is the constraint?
7	Hierarchies and Barriers	Can you prove that more time helps? And why does that not settle P versus NP?
8	Randomised Complexity	Can randomness make a problem tractable?

Wk	Module	Core question it answers
9	Circuits	If you allow a different program for each input length, does that help?
10	Interactive Proofs	If you may ask questions, what can be proved to you?
11	Hardness of Approximation	If you cannot solve it exactly, how close can you guarantee?
12	Fine-Grained and Parameterised	Not 'is it polynomial' but 'what exactly is the exponent, and in what'?
13	What Complexity Tells You	After thirteen modules, what has the classification bought?

Projects

Project 1 — A hardness proof, constructed (after Module 7)

Prove something NP-complete yourself, and place several problems correctly in the classes.

Requirements

- **One NP-completeness proof you constructed**, not reproduced — both directions, with the polynomial bound on the reduction verified.
- **The membership half done properly**, which is the half people skip: exhibit the certificate and the verifier.
- **Three problems placed** in P, NP-complete, or beyond, with justification.
- **One problem placed in co-NP** and the asymmetry explained.
- **One problem placed in PSPACE** with an argument for why it is probably not in NP.
- **A hierarchy theorem proved**, in your own words.

Done means

- **Both halves of the NP-completeness proof present** — membership and hardness. A hardness proof alone proves NP-hardness, not completeness, and the distinction is graded.
- **The reduction's polynomial bound actually argued**, not assumed.
- **The reduction in the correct direction**, stated explicitly.
- **And one problem you initially placed wrongly**, with the error and the correction written down.

Project 2 — A hard problem, handled (after Module 12)

Take an NP-hard problem you care about and establish what the theory actually predicts about solving it.

Requirements

- **An NP-hard problem from your own work or interest**, with its hardness established or cited.
- **A solver run on real instances**, with sizes and times reported.
- **An instance family where it fails**, and one where it succeeds easily, with the difference characterised.
- **A parameter identified** and the problem's fixed-parameter tractability assessed (Module 12).
- **The best approximation ratio known**, and the hardness-of-approximation result if one exists (Module 11).
- **A fine-grained lower bound** if one applies — SETH, 3SUM, or APSP-based.

Done means

- **The easy and hard instance families genuinely distinguished**, with a stated hypothesis about why.
- **The parameter assessed correctly** — including if the answer is that no useful parameter is small.
- **The approximation picture stated with both sides**: achievable ratio and hardness bound.

- **And a written answer to 'what does NP-hardness actually predict here?'** — which is the graded part, and 'less than I expected' is a common correct answer.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Arora & Barak — Computational Complexity: A Modern Approach (free draft)	The primary source, and the full draft is free from the authors. Chapters 1–11 and 17–20 are this course, and the barrier results of Modules 07 and 09 are treated better here than anywhere else.
Sipser — Introduction to the Theory of Computation, chapters 7–10	The gentler path through Modules 01–07 , and the same book as CSCE 627 — so the two courses share a text, which makes the computability/complexity boundary easy to follow.
MIT 18.404J — the complexity half (free lectures and problem sets)	Free video lectures covering Modules 01–07 , by Sipser, with graded problem sets and solutions.
Garey & Johnson — Computers and Intractability	The catalogue of NP-complete problems , and still the reference for Module 04's reductions. Its appendix of 300 problems is what you reduce from in practice. Library copy.
Cygan et al. — Parameterized Algorithms (free PDF)	Module 12's reference, free in full — the most practically useful material in the course, and the least commonly taught.
Aaronson — P =? NP (free survey)	The honest survey of the central question , including the barriers of Modules 07 and 09 and an assessment of what the evidence is actually worth. Read it twice.

Tooling

- **Paper, as in CSCE 627. The deliverable is proofs**, and a reduction you cannot write out is a reduction you have not got.
- **A SAT solver and a MIP solver** (CSCE 625 Module 08, CSCE 669 Module 13). **Module 04's theory is far more convincing once you have watched a solver dispatch a million-variable NP-complete instance.**
- **An instance generator**, so you can produce hard and easy instances of the same problem deliberately — which is Module 12's central observation made experimental.
- **A profiler and a plotting setup**, for Module 12's fine-grained work: **the difference between n^2 and $n^{2.5}$ is visible on a log-log plot and invisible in a table.**
- **Garey and Johnson's appendix, or a modern equivalent. Reductions are built from a library** (CSCE 627 Module 06 §3), and the library is the tool.
- **And no GPU, no cluster, no data. The second course in the program with no computational requirement**, which is worth noticing twice.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.